

Python Project Requirements

Project Scope

You will design and implement a non-trivial Python utility that performs system, file, or data analysis tasks using standard Unix command-line tools.

The project must demonstrate

- structured Python programming
- reasoning based on command exit status
- controlled parsing and reporting

Project Selection

You will design your own project. It may be best to discuss your project idea with the instructor before starting. Originality (uniqueness) will factor into your project grade.

Functional Requirements

1. Command-Line Interface

- Script must be invoked from the command line
- Must support at least 2 modes or subcommands
- Must provide a clear usage message when invoked incorrectly

2. Command-Line Arguments

- Script must take command-line arguments
- At least 1 required argument (mode)
- At least 3 optional arguments

3. Functions & Structure

- Minimum 6 functions
- 1 of the 6 is a main function
- 1 of the 6 is a usage function
 - called when script arguments are incorrect
- At least 2 of the 6 must:
 - take input (parameters)
 - return status only (0 or non-zero/ true or false)
 - do not print output
- Only main and usage functions may terminate the script
- Simple functions do not count towards the 6 required
 - An example: checking if something is a directory

- Helper functions only used once do not count towards the 6 required
- Exit Codes
 - Scripts must use the following exit codes:
 - 0 - Pass / success
 - 2 - Incorrect usage
 - 1 - Fail
 - Command Line Tools
 - Must use subprocess at least once
 - Python Capabilities vs Command Line Tools
 - Most simple command-line tools can be replicated with Python code
 - Use Python if the capabilities is built-in
 - Use command-line tools if capability not built-in to Python
 - Output Parsing & Manipulation
 - Must compute derived metrics using parsing pipelines
 - Examples:
 - counts
 - top-N lists
 - summaries
 - comparisons
 - Testability
 - Project must be fully testable on local machine
 - No privileged access (i.e have a user account on a remote)

Required Output

Each script must produce:

1. Human readable output
2. Optional report file

Deliverables

All deliverables will be submitted as a single zip file.

1. Python script
 - single .py file
2. README.md, must include:
 - Project description
 - Supported modes and options
 - Exit code meanings
 - How to test locally
 - [Markdown Guide](https://www.markdownguide.org/)  [\(https://www.markdownguide.org/\)](https://www.markdownguide.org/)

Code Defense

All students are required to participate in their assigned code defense to receive a grade for the project.

5 students will be chosen at random to publicly defend their code. For each student defending their code, 2 students will be randomly chosen to act as the panel. All other students will be required to do a one-on-one code defense.

Code defenses will take approximately 20 minutes. The defending student will have 10 minutes to explain their project/code. The panel members will spend 10 minutes asking questions about the project / code. Panel members will be required to submit their questions prior to the code defense. The questions will not be provided to the defending student.

Defending students will only be able to utilize the terminal to demonstrate their script functioning, and a comment free script file. No notes, or other materials.

Participation in your assigned code defense is worth 10% of the project grade. Defending students will be graded on demonstrating a thorough understanding of their code and Python programming techniques. Panel members will be graded on the quality of their submitted questions, and active participation during the defense.